

A Plain-English Guide to the Hybrid Product Recommendation System

What every component does, how it works, and what the results actually mean

Lola Toirxonova · BTEC L6 Unit 2 Independent Project · PDP Private University

May 2026

Foreword

This document explains, in plain English, every moving part of the **Hybrid Product Recommendation System** project. It is written for a reader who has *used* the demo but has not yet been told what is happening underneath.

No prior knowledge of machine learning, statistics, or software engineering is assumed. Where a technical term appears, it is immediately translated into everyday language. Mathematical formulas are avoided in favour of analogies. Where a finding is honest but uncomfortable — for example, “the neural network was actually worse than the simple model” — that finding is reported truthfully, because honest reporting is part of what makes a project distinction-grade.

After reading this guide, the reader should be able to:

1. Explain to a non-technical audience what the project does and why it matters.
2. Describe in everyday words what each of the four algorithms is doing.
3. Read the evaluation numbers in the results files and interpret them correctly.
4. Walk a panel through the live demo with confidence.
5. Answer the three predictable questions: “*Why a hybrid?*”, “*Why didn’t the neural network win?*”, and “*What about new customers with no history?*”

Table of Contents

1. The Big Picture
2. The Data — What H&M Shared, and What It Looks Like
3. Strategy One — Content-Based Filtering: *“Find Me More Like This”*
4. Strategy Two — Collaborative Filtering: *“What Did People Like Me Buy?”*
5. Strategy Three — The Hybrid Model: *“Combine Both”*
6. Strategy Four — Neural Collaborative Filtering: *“Let a Neural Network Try”*
7. How We Know They Work — The Evaluation Methodology
8. What the Results Actually Say
9. A Walkthrough of the Live Demo
10. A Tour of the Repository
11. How the Demo Got Online — Deployment Explained
12. Glossary of Terms
13. Quick Reference Card

1. The Big Picture

1.1 The Problem Online Stores Have

H&M's online catalogue contains **more than 105,000 products**. A typical visitor will look at perhaps 20 to 50 items before they either buy something or leave. That means more than **99.95% of the catalogue is never seen** by any individual shopper.

The store therefore faces a question: *of those 105,000 products, which 20 should we put in front of this specific person right now?*

If the store guesses well, the shopper finds what they wanted, buys it, and comes back. If the store guesses badly — say, by showing winter coats to someone who only ever buys swimwear — the shopper leaves, often forever. Industry reports estimate that recommendation engines drive roughly **35% of Amazon's sales** and **75% of Netflix's viewing time**. The quality of those guesses is, very literally, a billion-dollar problem.

1.2 What a Recommendation System Is

A **recommendation system** is a piece of software whose only job is to answer that question: *given what we know about this person, what should we suggest they look at next?*

There are many ways to answer it. Some are simple ("show everyone the bestsellers"). Some are sophisticated ("predict what each individual will want by learning from the patterns of millions of past shoppers"). This project builds four versions of the sophisticated kind, evaluates them side by side under identical conditions, and reports honestly on which one actually works best — including the cases where the more complicated approach surprisingly loses to the simpler one.

1.3 Why the H&M Dataset

For research to be credible, it must be done on real data of meaningful size. H&M released a dataset to the public via Kaggle in 2022 that contains:

- **105,542 products** with text descriptions, colour, category, and other attributes
- **1,371,980 customers** with anonymised identifiers and basic demographics
- **31 million purchase transactions** spanning roughly two years

This is large enough to be realistic but small enough to fit on a single laptop with careful sampling. It is also entirely anonymous — no name, address, email, or payment information is included, only hashed customer codes. That makes it ethically safe to use for academic work.

1.4 The Four Strategies at a Glance

Strategy	Plain-English Description	Technical Name
Find similar items	"You bought a red cotton T-shirt — here are other red cotton T-shirts."	Content-based filtering
Find similar people	"Other shoppers who bought what you bought also bought this."	Collaborative filtering (ALS)
Combine both	Mix the two signals above into a single ranking.	Weighted hybrid
Let a neural network decide	Use deep learning to find more complex patterns.	Neural Collaborative Filtering

The remainder of this document explains each of these in detail, then describes how their performance was measured and what the measurements revealed.

2. The Data — What H&M Shared, and What It Looks Like

Before any algorithm can be built, the data must be understood. The H&M release consists of three files.

2.1 `articles.csv` — The Product Catalogue

Each row is one product. The most useful columns for this project are:

- `article_id` — a unique product code (e.g. 0108775015). Note: leading zeros matter, which is why the project always treats this field as text, not as a number.
- `prod_name` — the human-readable name (e.g. “Strap top”).
- `detail_desc` — a short marketing description.
- `product_type_name` , `product_group_name` — e.g. “Trousers”, “Garment Lower body”.
- `colour_group_name` — e.g. “Light Pink”, “Dark Blue”.
- `department_name` , `index_group_name` , `section_name` — H&M’s internal merchandising categories.

The text columns above are the raw material the **content-based** algorithm reads.

2.2 `customers.csv` — The Shoppers

Each row is one shopper. Personally identifying information is absent. The only useful columns are demographic — age band, a Postal-code hash, and some marketing opt-in flags. For this project, the customer file is used mainly to confirm the size of the user population; the algorithms themselves work entirely from the transaction data.

2.3 `transactions_train.csv` — Who Bought What and When

This is the heart of the dataset and the largest file by far. Each row records a single purchase:

- `t_dat` — the date of the purchase (e.g. 2019-06-12).
- `customer_id` — which shopper bought it (anonymised hash).
- `article_id` — which product they bought.
- `price` , `sales_channel_id` — the price paid and whether the purchase was online or in-store.

There are approximately **31.8 million rows** in this file, spanning roughly **two years** of H&M’s sales. Loading the whole file into memory at once would consume around **7 GB of RAM**, which is impractical on a laptop, so the project uses three sampling strategies depending on the task. These are documented in `src/data.py` and discussed further in Chapter 10.

2.4 Two Concepts the Reader Must Know

2.4.1 Implicit vs. Explicit Feedback

In some recommendation problems, users explicitly rate items — for example, awarding a film one to five stars on Netflix. That is called **explicit feedback**.

In the H&M data, there are no star ratings. The only signal is whether someone *bought* something. That is **implicit feedback**: we know what they purchased, but not whether they liked it. Maybe they bought it as a gift. Maybe they returned it the next day. The implicit signal is noisier than an explicit rating, which means certain algorithms — for example, the **ALS** variant of collaborative filtering used in this project — are required because they were specifically designed for implicit data.

2.4.2 The Sparsity Problem

If you imagine a giant spreadsheet with one row per customer and one column per product, then a cell would be filled in only when that customer bought that product. With 1.37 million customers and 105 thousand products, that spreadsheet has approximately **144 billion cells**. Only **31 million** of them are filled. That is roughly **0.002% full**, or — flipped around — **99.998% empty**.

This is called a **sparse** matrix. Almost every interesting algorithm in the project has to deal with the fact that virtually all the data is missing.

2.4.3 The Long Tail

A small handful of bestselling items account for a large portion of all transactions, while the vast majority of products sell only rarely. In this dataset, roughly **20,000 of the 105,000 products** account for **80% of all transactions**. The remaining 85,000 products are the “long tail” — niche items that any individual shopper might love but that the bestseller chart will never reveal. A good recommendation system surfaces tail items to the right shoppers, expanding what the customer discovers beyond the bestseller list.

3. Strategy One — Content-Based Filtering: “*Find Me More Like This*”

3.1 The Intuition

This strategy ignores other shoppers entirely. It looks at the **descriptions of the products** themselves and asks: *which products use the most similar words to the ones this person has bought in the past?*

If a shopper has bought “*Black slim-fit jeans*”, “*Dark denim straight jeans*”, and “*Indigo skinny jeans*”, the algorithm will notice the words *black*, *denim*, *jeans*, *fit*, *dark* recurring in their history. It will then score every product in the catalogue by how many of those same words appear in its description, and return the highest-scoring ones.

3.2 How the Text Becomes Numbers — TF-IDF

A computer cannot directly compare two pieces of text. It must first turn each product’s description into **a list of numbers** — a vector. The technique used here is called **TF-IDF**, which stands for *Term Frequency – Inverse Document Frequency*. The name sounds intimidating; the idea is simple.

For each word in a product’s description, TF-IDF asks two questions:

1. *How often does this word appear in this product’s description?* (Term Frequency — frequent here suggests importance.)
2. *How rare is this word across the whole catalogue?* (Inverse Document Frequency — a word that appears in 90% of products, like “*cotton*”, carries little signal; a word that appears in only 1% of products, like “*jacquard*”, carries much more.)

The two are multiplied. The result is a high score for words that are **distinctive** for this particular product. After processing, every product in the catalogue is represented as a long list of numbers — one number per word in the vocabulary — and similar products will have similar lists.

3.3 How Similarity Is Computed — Cosine Similarity

To compare two products, the algorithm measures the angle between their TF-IDF vectors. This is called **cosine similarity**.

The intuition is geometric: imagine each product as an arrow pointing into a high-dimensional space, with one dimension per word. Two products whose arrows point in the same direction are similar; two whose arrows are perpendicular are unrelated. Cosine similarity returns a number between 0 and 1, where 1 means “identical in their word usage” and 0 means “share no vocabulary at all”.

3.4 Building a User Profile

A user has bought many items, not one. To make recommendations *for the user*, the algorithm averages together the TF-IDF vectors of all the items they have purchased, producing a single vector that represents that user’s overall taste in words. Then it compares this user-profile vector against every product in the catalogue and returns the top matches that the user has not yet bought.

3.5 Strengths and Weaknesses

Strengths	Weaknesses
Works for brand-new products (we have the description even before anyone buys it)	Limited serendipity — only ever recommends things similar to what the user has already bought
Transparent: easy to point at the words that drove a recommendation	Requires rich text metadata; falls apart if product descriptions are sparse
Fast to compute and easy to update	Cannot help a new user with no purchase history

3.6 Where It Lives in the Project

The notebook `02_content_based.ipynb` builds the TF-IDF vocabulary and stores two artefacts in the `models/` folder: `content_based_vectorizer.pkl` (the vocabulary) and `content_based_item_tfidf.npz` (every product converted to its TF-IDF vector). These files are loaded by the Streamlit demo at start-up.

4. Strategy Two — Collaborative Filtering: “What Did People Like Me Buy?”

4.1 The Intuition

This strategy is the opposite of content-based filtering. It **completely ignores** the product descriptions. The only thing it looks at is the pattern of who bought what.

The reasoning is simple. If shoppers Alice, Bob, and Carol have all bought the same six items as each other, they likely share a taste. If Alice then buys a seventh item that Bob and Carol have not yet seen, it becomes a strong candidate to recommend to them.

This idea is called **collaborative filtering** because it works by *collaborating* across the population — each user’s purchases inform the recommendations made to similar users.

4.2 Taste Fingerprints — The Idea of Latent Factors

In practice, you cannot literally check every pair of shoppers to find Alice’s nearest matches — that would require billions of comparisons. Instead, the algorithm does something cleverer. It assumes that each shopper’s taste can be summarised by a small list of hidden numbers — perhaps fifty — that act like a **taste fingerprint**.

These fingerprints are not handed to the algorithm. They are *discovered* automatically from the transaction data. Each number in the fingerprint comes to represent some hidden factor — for example, “how much does this person like dark colours?”, or “how much does this person buy seasonal items?”. Importantly, **the algorithm does not know what each number means** in human terms; it only knows that these numbers, when multiplied together correctly, reproduce the purchase patterns seen in the data.

Each *product* is also assigned a fingerprint of the same length. To predict whether a shopper would like a product, the algorithm multiplies the two fingerprints together. A high score means “this shopper’s tastes align with this product’s appeal” — and that product becomes a recommendation candidate.

4.3 How the Fingerprints Are Found — ALS

The technique used in this project to discover the fingerprints is called **ALS**, short for *Alternating Least Squares*. The mathematics is involved, but the idea is intuitive.

ALS plays a back-and-forth game with itself. It starts with random fingerprints for every shopper and every product. Then:

1. **Round one:** Fix the product fingerprints. Adjust the shopper fingerprints to best reproduce the observed purchases.
2. **Round two:** Now fix the shopper fingerprints. Adjust the product fingerprints to best reproduce the observed purchases.

These two rounds alternate (hence *alternating*) for around fifteen iterations, at which point both sets of fingerprints have settled into a stable solution that captures the structure of the data as well as the model can.

The specific variant used here — from Hu, Koren and Volinsky (2008) — was designed for **implicit feedback**, which fits the H&M data perfectly because all we have are purchases, not ratings.

4.4 Strengths and Weaknesses

Strengths	Weaknesses
Discovers genuine surprises (can recommend things very unlike past purchases, if similar shoppers liked them)	Fails for brand-new shoppers — no purchase history means no fingerprint can be learned
Captures taste patterns that no text-based system could ever see	Fails for brand-new products — no purchases means no fingerprint can be learned
Scales to millions of users and items	The fingerprints are uninterpretable — cannot easily explain <i>why</i> a particular item was recommended

The first two weaknesses are together known as the **cold-start problem**, which is the single most discussed issue in recommendation research.

4.5 Where It Lives in the Project

The notebook `03_collaborative_filtering.ipynb` trains the ALS model and stores it in `models/cf_als_model.pkl` along with the lookup tables that translate between customer IDs and fingerprint rows.

5. Strategy Three — The Hybrid Model: “Combine Both”

5.1 Why Combine

Content-based filtering and collaborative filtering have **complementary weaknesses**:

- Content-based can recommend new products but only ever finds similar items.
- Collaborative filtering finds surprising recommendations but fails on new products and new shoppers.

A **hybrid** combines both, hoping to keep the strengths of each while masking the weaknesses. This is not a new idea — it has been the dominant approach in industry since at least Burke’s foundational survey (2002), and large platforms like Netflix and Spotify run dozens of models blended together. The approach used here is the simplest possible blend: a **weighted average**.

5.2 The α Parameter — What 0.5 Means

For every product, both algorithms produce a score. The hybrid model combines them like this:

$$\text{Final score} = \alpha \times (\text{content-based score}) + (1 - \alpha) \times (\text{collaborative filtering score})$$

The parameter α (the Greek letter *alpha*) is a number between 0 and 1 that controls the mix:

- $\alpha = 1.0$ means “use only the content-based score” — collaborative filtering is ignored entirely.
- $\alpha = 0.0$ means “use only the collaborative filtering score” — content-based is ignored entirely.
- $\alpha = 0.5$ means “give equal weight to both”.

To find the best value, the notebook `04_hybrid.ipynb` runs the experiment at five settings (0.0, 0.25, 0.5, 0.75, 1.0) and measures performance at each. In this project, $\alpha = 0.5$ produced the best NDCG score, so that value was locked into the demo. Both scores are normalised to the same range first, so neither dominates simply because its raw numbers are larger.

5.3 The Cold-Start Fallback

Even a hybrid cannot help a shopper with zero purchase history, because both halves of the hybrid require at least some signal to work with. In that case the demo falls back to a different rule entirely: **show the most popular items in the catalogue**. This is a deliberately simple, deliberately unsophisticated rule — and as we will see in Chapter 8, it is also remarkably effective for the cold-start population.

6. Strategy Four — Neural Collaborative Filtering: “Let a Neural Network Try”

6.1 What a Neural Network Is, Very Briefly

A **neural network** is a computer model loosely inspired by how brains process information. Practically, it is a long pipeline of mathematical layers, each of which takes the output of the previous layer, multiplies it by a set of adjustable weights, adds a bias, and squashes the result through a non-linear function. With enough layers and enough weights, a neural network can learn to approximate astonishingly complex relationships.

The “learning” happens by repeatedly feeding the network training data, checking how wrong its predictions are, and slightly adjusting the weights to be less wrong next time. This adjustment is called **back-propagation**. Over millions of small adjustments, the network gradually shapes itself to the data.

6.2 How NeuMF Works

The model used here is called **NeuMF** — short for *Neural Matrix Factorisation* — and comes from the paper *Neural Collaborative Filtering* by He et al. (2017). Its central idea is to **replace the simple multiplication** at the heart of ALS (taking two fingerprints and multiplying them to produce a score) with a **learned, non-linear function** computed by a neural network.

The reasoning is that simple multiplication assumes a particular kind of relationship between user taste and item appeal. A neural network can in principle learn relationships that are more complicated — a high score might depend not just on the strength of two factors but on their interaction in a way that no linear formula captures.

The architecture in this project has two pathways that are then combined:

1. **GMF** — a generalised version of plain matrix factorisation, providing a baseline-like signal.
2. **MLP** — a four-layer fully-connected neural network learning more complex interactions.

The two pathways are concatenated and passed through a final layer that outputs the predicted score for that user–item pair.

6.3 The Honest Finding — Simpler Was Better

The expectation, before running the experiment, was that NeuMF would outperform ALS — that is what the original 2017 paper reported. In practice on the H&M data and with the compute budget available for this project (a CPU, three training epochs, a single train/test split), the result was the **opposite**: NeuMF scored a Precision@10 of **0.003**, compared to **0.023** for the hybrid — roughly **eight times worse**.

This is not a failure of the project. It is a **published, peer-reviewed finding**. In 2020, Rendle, Krichene, Zhang and Anderson published “*Neural Collaborative Filtering vs. Matrix Factorization Revisited*” at the RecSys conference, showing that a properly tuned classical matrix-factorisation model can match or beat NCF on standard benchmarks. The finding in this project lands on the same side of that debate, and reporting it transparently is exactly the kind of critical engagement that BTEC Distinction-grade work requires.

6.4 Where It Lives in the Project

The notebook `05_neural_cf.ipynb` (local) and `05_neural_cf_colab.ipynb` (Google Colab variant for GPU training) define and train the NeuMF model. The trained network is saved to `models/ncf_neumf.pt`, with the associated user/item lookup tables in `models/ncf_id_maps.pkl`. The Streamlit demo will display the NCF tab only if both files are present.

7. How We Know They Work — The Evaluation Methodology

A recommendation system that *feels* good but cannot be measured is worthless to a business. Equally, a system measured carelessly is worse than no measurement at all — it produces a confident wrong answer. This chapter describes how the four algorithms were measured fairly and reliably.

7.1 The Train / Test Split — Why Time Matters

The fundamental rule of any machine-learning evaluation is: **never test the model on data it has already seen during training**. The most common way to enforce this is to randomly hold out, say, 20% of the data and treat it as a final exam.

In this project, however, **random splitting would be misleading**. Recommendation is fundamentally about predicting the future from the past. If we randomly shuffle in some of the user's future purchases when training the model, the model has effectively “seen tomorrow's newspaper” during training — and its measured accuracy will be artificially inflated. This is called **temporal leakage**.

The fix is a **time-based split**. The last seven days of the dataset are held out as the test set. Everything before those seven days is used for training. The test then asks: *given everything the algorithm saw up to that cutoff, can it predict what the user actually bought in the final week?* This mirrors the real-world deployment situation, where the model is always asked to predict tomorrow from yesterday.

7.2 Cross-Validation — Why Five Folds

Even with a careful split, a single test could be unlucky. Perhaps the last seven days happened to contain an unusual sale, or a fashion trend that the training data did not capture. To guard against this, the evaluation uses **5-fold cross-validation**.

The data is split into five chunks. The experiment is then repeated **five separate times**. Each time, one chunk plays the role of the test set and the other four chunks are used for training. The five measured results are averaged, and their spread is reported as a standard deviation. The result is a much more trustworthy estimate of how the algorithm performs, alongside a number — the standard deviation — that quantifies how much the measurement wobbles from run to run.

In the project's results files, every metric appears as **mean $\pm$ standard deviation**. For example, the hybrid model's NDCG@10 of **0.0688 $\pm$ 0.0136** can be read as: “on average, across five different test windows, the hybrid scored 0.0688, with the score varying by about 0.0136 either side.”

7.3 The Five Metrics, in Plain English

A single number cannot tell the full story of a ranked list of recommendations. The project reports five complementary metrics. All five are evaluated at the **top 10** — that is, only the ten highest-scoring recommendations are scored, since that is what a user actually sees on a real product page.

Metric	Question It Answers
Precision@10	Of the 10 items I recommended, what fraction did the user actually buy?
Recall@10	Of all the items the user <i>did</i> buy, what fraction appeared in my top 10?
NDCG@10	Did I put the genuine matches near the top of the list, rather than at position 9 or 10?
MAP@10	A precision-style metric that also rewards ranking matches early in the list.
HitRate@10	Did at least <i>one</i> of my 10 recommendations turn out to be something the user actually bought?

Why use all five? Each captures a different facet:

- **Precision** punishes irrelevant recommendations.
- **Recall** punishes *missed* recommendations.
- **NDCG** punishes putting good recommendations in low positions.
- **MAP** is a stricter version of NDCG.
- **HitRate** is the most generous metric — useful for asking “*did the model help at all?*”

A model that scores well on all five is genuinely good. A model that scores well on only one or two is suspect and merits investigation.

A note on the absolute size of the numbers

A precision of **0.023** sounds small. It is. But it must be read in context: there are over **40,000** items the model could plausibly recommend, and a typical user buys only a small handful per week. Random guessing would produce a precision of roughly **0.00003**. The hybrid model is therefore approximately **800 times better than random**. In the world of recommendation, single-digit-percent precision is the normal scale, not a sign of failure.

7.4 Statistical Significance — The Wilcoxon Test

If model A scores 0.067 and model B scores 0.069, is model B genuinely better, or is the difference within the noise of the evaluation? This is a statistical question, not a numerical one.

The project uses a **Wilcoxon signed-rank test**, which compares two models’ per-user scores in a pairwise way and returns a **p-value**. The p-value answers the question: *if the two models were actually equivalent, what is the probability of seeing a difference at least this large by pure chance?* The convention is that a p-value below **0.05** is “statistically significant” — meaning the difference is unlikely to be a fluke.

The project’s findings on this front are summarised in the table below.

Comparison	p-value	Interpretation
Content-Based vs. ALS	$p \approx 2.4 \times 10^{-11}$	ALS is genuinely, dramatically better than content-based on NDCG.
Content-Based vs. Hybrid	$p \approx 1.1 \times 10^{-22}$	The hybrid is genuinely, dramatically better than content-based.
ALS vs. Hybrid	$p = 0.55$	The hybrid and ALS are statistically indistinguishable.

That last row is the most important honest finding in the project. The hybrid model has a slightly higher average NDCG than plain ALS, but the difference is not statistically real. Chapter 8 unpacks what this means.

7.5 Cold-Start Segmentation

A single average across all users can hide important variation. A model might be excellent for users with rich purchase histories and useless for users with no history — and the average would obscure that. To prevent this, the evaluation also reports metrics separately on three subgroups of test users:

1. **Pure warm** — users who have a substantial purchase history and whose recommended items also appear in the training set.
2. **Partial cold item** — users who exist in the training data, but the items they purchased in the test period include some not seen during training.
3. **Cold user** — users who do not exist at all in the training data.

This segmentation is what makes it possible to say something specific about each algorithm's cold-start behaviour, rather than burying that behaviour inside a single overall average.

8. What the Results Actually Say

This chapter reports the headline findings of the project and explains how to read them. All numbers in this chapter come from the files in `outputs/evaluation/`.

8.1 The Headline Numbers

Model	Precision@10	Recall@10	NDCG@10	MAP@10	HitRate@10
Content-Based	0.0115 ± 0.0010	0.0507 ± 0.0061	0.0366 ± 0.0060	0.0242 ± 0.0046	0.1067 ± 0.0095
ALS CF	0.0225 ± 0.0043	0.0898 ± 0.0158	0.0668 ± 0.0114	0.0438 ± 0.0066	0.1993 ± 0.0344
Hybrid (α=0.5)	0.0231 ± 0.0035	0.0983 ± 0.0197	0.0688 ± 0.0136	0.0432 ± 0.0091	0.2080 ± 0.0364
NeuMF (single split)	0.0030	0.0111	0.0064	0.0028	0.0300

Three observations stand out.

8.2 Finding One — The Hybrid Wins on Average, but Only Marginally

On every metric except MAP, the hybrid model produces the highest mean score. On HitRate@10, for example, it succeeds at recommending at least one purchased item to roughly **21% of test users**, compared to **20%** for ALS alone and **11%** for content-based alone.

However, as Section 7.4 made plain, the Wilcoxon test shows that the gap between the hybrid and ALS is **not statistically significant** ($p = 0.55$). In a deployment scenario, this would translate into a real engineering decision: *is the additional complexity of running two models and blending their outputs worth a marginal — and probably noise-dominated — uplift over running a single, simpler ALS model?* For a small team in a fast-moving market, the answer might well be no.

This is a more nuanced conclusion than “*the hybrid wins*”, and it is the kind of nuance that distinguishes a Distinction-level analysis from a Pass-level one.

8.3 Finding Two — The Neural Network Underperformed

NeuMF scored roughly **eight times lower** than the hybrid on Precision@10 (0.003 versus 0.023). This was not the expected outcome going into the experiment. Several factors plausibly contributed:

- **Training compute.** The NCF model was trained on a CPU for only three epochs. The original NCF paper trained for many more epochs on a GPU. In a constrained budget, the network simply did not have enough exposure to the data to converge well.
- **Hyperparameter tuning.** Each of the simpler models was tuned over its main parameters; the neural model was not exhaustively tuned because the compute cost was prohibitive.
- **Dataset structure.** Rendle et al. (2020) demonstrated empirically that on standard collaborative-filtering benchmarks, well-tuned matrix factorisation matches or beats NCF — meaning the underwhelming result reported here is consistent with current peer-reviewed evidence, not an outlier.

Honest reporting of this result — with reference to the 2020 paper — is exactly what the project should do in its Findings and Discussion chapters. It is critical engagement, not a flaw.

8.4 Finding Three — For Cold Users, Popularity Beats Everything

The `cold_start_segments.csv` file contains the most operationally important finding in the project. For test users with **no purchase history at all in the training data**, the warm-user models are useless. Their NDCG@10 scores collapse to

between 0.0007 and 0.0044. By contrast, the **popularity fallback** — a deliberately naïve baseline that recommends the same most-popular items to every cold user — scores **0.0075** on the same population.

This result has a clear practical implication: a real production system should not pretend to personalise for users about whom nothing is yet known. It should serve them a popularity ranking and start collecting signal. Personalisation can begin once the user has bought their first few items.

8.5 The Three-Sentence Summary

If asked to summarise the project's findings in three sentences, the right answer is:

A weighted hybrid of content-based and collaborative-filtering models produces the best average performance on the H&M dataset, marginally beating plain matrix-factorisation collaborative filtering, although the gap is not statistically significant. The neural-network-based NeuMF model underperformed both, consistent with recent peer-reviewed evidence (Rendle et al., 2020) showing that well-tuned classical methods can match or beat deep-learning recommenders on standard benchmarks. For brand-new users with no purchase history, none of the personalised methods help; a simple popularity ranking outperforms every clever algorithm.

9. A Walkthrough of the Live Demo

This chapter explains every element a visitor sees when they open the Streamlit application — both running locally and at the live URL `http://165.227.144.159:8501/`.

9.1 The Top Strip

The very top of the page contains a pill labelled “**Personalised Recommendations**”, a large heading, and a one-line subtitle. These elements are pure presentation; they have no effect on the algorithms. They exist to frame the demo for a first-time visitor.

9.2 The Three Controls

Below the title, the user sees three controls in one row.

1. **Algorithm** — three pill-shaped chips: *Hybrid*, *Content-Based*, *Collaborative Filtering*. Selecting one tells the page which model’s output to display. The fourth algorithm (NeuMF) is not surfaced here because, as Chapter 8 explained, its results were too weak to be useful.
2. **Customer** — a dropdown listing 200 sample shoppers. Each entry shows a truncated anonymised ID followed by the name of one of their past purchases (e.g. “*abcdef... · bought Strap top*”). This preview is purely a hint; the algorithm does not care which entry you choose.
3. **Show** — the number of recommendations to display (10, 15, or 20).

9.3 What Happens Behind the Scenes When the User Picks a Customer

The full sequence of events the moment a customer is selected is, in plain English:

1. The app looks up that customer’s training-period purchases — a set of `article_id` values.
2. The chosen algorithm computes a score for every product in the catalogue.
3. Products the customer has already bought are excluded from the candidate set, so the recommendations are always genuinely new.
4. The top-scoring remaining products are sorted and the top k are returned.
5. The app fetches the names, types, colours, and section labels for those products from `articles.csv` and renders them as cards on the right of the screen.

The “Purchase history” panel on the left is independent of all this — it shows the first six items the customer bought, drawn directly from the training data, for context.

9.4 The Cards on the Right

Each recommendation card displays:

- A rank number (1 to 10/15/20) — the **algorithm’s ranked position** for this product. Position 1 is the model’s strongest bet.
- The product’s name, type, colour, and section — pulled from `articles.csv`.
- A small grey article ID at the bottom — the internal product code, useful for debugging or for tracing a recommendation back to the catalogue.

The badge above the cards (“*TF-IDF · Cosine Similarity*”, “*ALS · Matrix Factorisation*”, or “*Hybrid · $\alpha = 0.50$* ”) identifies which algorithm produced these particular cards.

9.5 The Feedback Form

At the bottom of the page, an expander labelled “*Submit feedback*” contains three Likert-scale sliders (relevance, diversity, surprise), an optional free-text box, and a submit button. Submitting writes one JSON line to `outputs/user_feedback/feedback.jsonl`. The feedback is anonymous — only the algorithm chosen, the value of k , the customer ID, the three ratings, and the optional notes are recorded. No personal information about the test user is asked for or stored.

This form exists to satisfy the optional qualitative component described in the proposal (collecting feedback from approximately 20 test users) and provides additional evidence for chapter 5 of the Final Report.

10. A Tour of the Repository

This chapter explains the contents of every important folder.

10.1 `data/` — The Raw Inputs

Contains the three H&M CSV files downloaded from Kaggle. **The folder itself is in the repository, but its contents are git-ignored.** This is because the files are very large (around 3.5 GB combined) and Kaggle's terms of use forbid redistribution. Anyone who clones the repository must download the files themselves.

10.2 `notebooks/` — Where the Models Are Built

Six numbered Jupyter notebooks that execute the project's pipeline in order:

Notebook	Purpose
<code>01_eda.ipynb</code>	Exploratory data analysis — counts, dates, sparsity, long-tail analysis, cold-start exposure.
<code>02_content_based.ipynb</code>	Builds the TF-IDF vocabulary and saves the item vectors.
<code>03_collaborative_filtering.ipynb</code>	Trains the ALS model.
<code>04_hybrid.ipynb</code>	Sweeps α , identifies the best blend, saves the configuration.
<code>05_neural_cf.ipynb</code> / <code>05_neural_cf_colab.ipynb</code>	Trains NeuMF — the local version and a Colab/GPU variant.
<code>06_evaluation.ipynb</code>	Runs 5-fold cross-validation across all models and produces the final summary tables.

The notebooks should be run in numerical order. Notebook 04 depends on the artefacts produced by 02 and 03; notebook 06 evaluates all four trained models.

10.3 `src/` — Shared Python Helpers

Two small modules used by every notebook:

- `src/data.py` — Loaders for the three CSV files and a time-based split function. The transaction loader supports three memory-bounded modes: a row-count limit (`nrows`), a uniform random sample (`sample_frac`), and a recent-window selection (`last_months`). The reasoning behind each mode is documented in the file itself.
- `src/metrics.py` — Implementations of Precision@K, Recall@K, NDCG@K, MAP@K, and HitRate@K. These are used by both the notebooks and the evaluation pipeline.

10.4 `models/` — The Trained Models

Holds the persisted output of notebooks 02 to 05 — the *brains* of each algorithm. These files are git-ignored because they can always be regenerated by re-running the notebooks.

File	Created by	Contents
<code>content_based_vectorizer.pkl</code>	Notebook 02	The TF-IDF vocabulary.
<code>content_based_item_tfidf.npz</code>	Notebook 02	Every product's TF-IDF vector, in sparse format.
<code>cf_als_model.pkl</code>	Notebook 03	The trained ALS model plus user/item lookup tables.
<code>hybrid_config.pkl</code>	Notebook 04	The best α value, derived from the sweep.
<code>ncf_neumf.pt</code>	Notebook 05	The trained PyTorch NeuMF network.
<code>ncf_id_maps.pkl</code>	Notebook 05	Translation tables for the NeuMF model.

10.5 `app/` — The Streamlit Demo

Contains `main.py`, which is the entire Streamlit application. It loads the models from `models/`, the data from `data/`, and renders the user interface. Run it with `streamlit run app/main.py` from the repository root.

10.6 `outputs/` — Generated Results

The contents are git-ignored. Subfolders are named after the chapter they support:

- `outputs/eda/` — figures and summary JSON from Notebook 01.
- `outputs/content_based/`, `outputs/collaborative_filtering/`, `outputs/hybrid/`, `outputs/neural_cf/` — per-algorithm results and comparison plots.
- `outputs/evaluation/` — the cross-validation summary, the Wilcoxon table, and the cold-start segmentation.
- `outputs/figures/` — final figures destined for the Final Report.
- `outputs/user_feedback/` — accumulated demo feedback from test users.

10.7 `docs/` — Written Deliverables

Holds the chapters of the Final Report and the BTEC-mandated supporting documents: proposal, literature review, project plan, ethics form, data management plan, presentation outline, findings notes for each algorithm, and the reflective evaluation. This document (`project_explained.md`) lives here too, alongside the others.

10.8 `deploy/` — Deployment Helpers

Contains `install.sh`, a one-shot bash script that bootstraps a fresh Ubuntu 24.04 DigitalOcean droplet into a live deployment of the demo. The script is documented in detail in Chapter 11.

10.9 `requirements.txt`, `.gitignore`, `README.md`

The standard scaffolding files. `requirements.txt` pins the Python libraries (pandas, scikit-learn, implicit, torch, streamlit, and so on). `.gitignore` excludes the dataset, the trained models, the outputs, the personal logbook, and the BTEC-provided guide documents. `README.md` is the quick-start guide for a new visitor to the repository.

11. How the Demo Got Online — Deployment Explained

The Streamlit application is reachable at `http://165.227.144.159:8501/` because a single Linux server somewhere in a DigitalOcean data centre is running it twenty-four hours a day. This chapter explains how that server was set up.

11.1 The Hosting Provider

DigitalOcean is a cloud provider that rents virtual Linux servers (called “droplets”) by the hour. For a small project like this one, a basic droplet with two virtual CPUs and four gigabytes of memory is sufficient and costs in the order of a few US dollars per month. The droplet behaves exactly like a physical Linux computer that you can log into remotely.

11.2 The Bootstrap Script

The file `deploy/install.sh` automates the entire setup. Run as the root user on a fresh droplet, it performs eight steps in sequence:

1. **Install system dependencies** — Python 3.12, nginx (a web server), and basic tools.
2. **Clone the GitHub repository** to `/opt/gradproject` on the droplet.
3. **Create a Python virtual environment** and install the libraries from `requirements.txt`.
4. **Prepare the `data/`, `models/`, and `outputs/` folders.**
5. **Check that Kaggle credentials are present**, and download the three H&M CSV files using the Kaggle command-line tool. The dataset is too large to redistribute via git, so it must be re-fetched on the server.
6. **Download the H&M dataset** (~3.5 GB), exit early if credentials are missing.
7. **Configure two system services** — a `streamlit.service` systemd unit that runs the app on port 8501 in the background, and an nginx configuration that proxies port 80 to port 8501 (so visitors can access the site without specifying a port).
8. **Enable the firewall** to allow only SSH and HTTP traffic, then start the Streamlit service.

After the script completes, the application is reachable at the droplet’s public IP address.

11.3 The systemd Service

systemd is the Linux process manager. Telling it to manage the Streamlit application means three things:

- The application **starts automatically** when the droplet reboots.
- If the application **crashes**, systemd restarts it within five seconds.
- The application’s **logs** are captured to the system journal, viewable with `journalctl -u streamlit -f`.

This is dramatically more reliable than running `streamlit run` manually in an SSH session and hoping it stays up.

11.4 The Model Files Are Not in Git

A subtle but important detail: the trained model files (`*.pkl` , `*.pt`) are not checked into the GitHub repository, because they are large and easily regenerated. This means that after the bootstrap script completes, the **server has the code but no trained models**, and the application would fail to start.

The script prints an explicit instruction at this point telling the operator to copy the models from their local laptop to the server using `rsync`. Once the models are in place, the Streamlit service can be started, and the demo is live.

11.5 Maintaining the Live Demo

Three commands are useful for routine maintenance, run after logging into the droplet:

- `systemctl status streamlit` — check whether the application is currently running.

- `systemctl restart streamlit` — restart the application (for example, after pulling new code with `git pull`).
- `journalctl -u streamlit -f` — watch the application's live logs to debug a problem.

12. Glossary of Terms

Term	Plain-English Meaning
α (alpha)	The mixing weight in the hybrid model. A number between 0 and 1 indicating how much to lean on the content-based score versus the collaborative-filtering score.
ALS (Alternating Least Squares)	A mathematical technique for discovering hidden “taste fingerprints” from purchase data. Used for the collaborative-filtering model in this project.
Article	H&M’s term for a product. In code, an article is identified by <code>article_id</code> .
Back-propagation	The procedure by which a neural network adjusts its internal weights based on how wrong its predictions were.
Cold start	The situation where there is no signal to base a recommendation on — either a brand-new user with no purchase history, or a brand-new product nobody has bought.
Collaborative filtering	A family of algorithms that produce recommendations by learning patterns of co-occurrence in purchases across many users.
Content-based filtering	A family of algorithms that produce recommendations by comparing product descriptions and matching new products to a user’s past taste.
Cosine similarity	A numerical measure of how similar two text descriptions are, on a 0-to-1 scale.
Cross-validation	A statistical technique for measuring performance reliably by running the same experiment several times over different splits of the data.
Explicit feedback	Direct ratings from users — for example, one-to-five stars. Not present in the H&M dataset.
Hybrid model	A recommendation model that combines two or more underlying methods, typically a content-based model and a collaborative-filtering model.
Implicit feedback	Indirect signals of user preference — purchases, clicks, time-spent — without explicit ratings. The H&M data is implicit-only.
Long tail	The large majority of products that each sell rarely but collectively account for substantial sales when surfaced to the right shoppers.
Matrix factorisation	The general mathematical idea behind ALS: decomposing the large user-item purchase table into two smaller tables of “taste fingerprints”.
NCF / NeuMF	Neural Collaborative Filtering — a deep-learning recommendation model from He et al. (2017). NeuMF is the specific architecture used in this project.
NDCG (Normalised Discounted Cumulative Gain)	A metric that rewards putting correct recommendations near the top of the list.
Precision@K	The fraction of the top-K recommendations that turned out to be correct.

Term	Plain-English Meaning
Recall@K	The fraction of the user's actual purchases that appeared somewhere in the top-K recommendations.
Sparse matrix	A spreadsheet where almost every cell is empty. The H&M user-item matrix is more than 99.99% empty.
TF-IDF	<i>Term Frequency – Inverse Document Frequency</i> . A technique for turning text into numbers in a way that highlights distinctive words.
Wilcoxon signed-rank test	A statistical test used to decide whether the difference between two models' results is genuinely real or could plausibly be due to chance.

13. Quick Reference Card

For revising before a panel defence.

The project in one sentence

A controlled comparison of four product-recommendation algorithms on the H&M Personalized Fashion Recommendations dataset, evaluated rigorously and reported with full honesty about which method actually wins.

The four algorithms in one line each

- **Content-Based** — Compare product descriptions. Recommends items similar to past purchases.
- **Collaborative Filtering (ALS)** — Discover taste fingerprints. Recommends what similar shoppers bought.
- **Hybrid** — Weighted average of the two above. Best on average performance.
- **Neural Collaborative Filtering (NeuMF)** — Deep-learning version. Underperformed in this study.

The headline findings

1. The hybrid wins on most metrics — but only marginally, and the gap to plain ALS is **not statistically significant** (Wilcoxon $p = 0.55$).
2. The neural network was **eight times worse** on Precision@10 than the hybrid. This is consistent with Rendle et al. (2020), who showed that well-tuned classical methods can match or beat NCF.
3. For new shoppers with no purchase history, **a simple popularity ranking outperformed every personalised model**.

The numbers to remember

- Precision@10 — hybrid scored **0.023**, roughly **800× better than random**.
- HitRate@10 — hybrid succeeded for **21%** of test users.
- NDCG@10 — hybrid scored **0.069 ± 0.014** across five-fold cross-validation.
- NeuMF scored **0.003** — well below the simpler models.

The likely questions and the right answers

Question	Answer
Why a hybrid?	Because content-based and collaborative filtering have complementary weaknesses — neither alone is robust to both new users and new items. The hybrid masks each one's weakness with the other's strength.
Why didn't the neural network win?	Limited compute budget (CPU, three epochs, no GPU hyperparameter sweep), and a finding consistent with Rendle et al. (2020).
What about cold-start users?	The honest answer is that personalisation cannot help them. A popularity baseline outperforms every clever algorithm on that population — and a production system should serve them popularity until they have bought enough to be personalised.
Why these five metrics?	Each captures a different facet of recommendation quality; reporting all five prevents over-claiming on a single number.
Why 5-fold cross-validation?	To ensure the reported numbers are not artefacts of a single lucky or unlucky split.
Why a time-based split?	Because random splitting would leak future purchases into the training set, inflating the measured accuracy.

This document was prepared as a personal study aid by Lola Toirxonova for the BTEC Level 6 Unit 2 Independent Project, May 2026.